

Implementation Brief

Minimal prototype specification | Markdown is the source of truth

Objective

Build the simplest production-usable prototype for 10-50 users. Optimize for a reliable core loop, fast mobile use, and easy deployment - not scale or architectural sophistication.

Required architecture

- UI and app generation: Lovable.
- Frontend: use Lovable's generated React stack; do not rewrite the framework without a clear blocker.
- Backend services: Supabase Auth, PostgreSQL, generated APIs, and Row Level Security.
- Hosting: deployable to Vercel.
- Separate Node, Express, Nest, Java, or other backend server: prohibited for this prototype.
- Secrets: never place a Supabase service-role key in browser code. The public anonymous key is acceptable only with correct Row Level Security.

Simplicity constraints

- Use the existing Lovable/Supabase client and component patterns.
- Keep dependencies minimal and avoid abstraction layers for hypothetical growth.
- Use direct Supabase queries from the app where Row Level Security safely permits them.
- Do not store derived daily counts, percentages, streaks, or heatmap cells.
- Do not add real-time subscriptions unless the basic query-and-refresh behavior is already correct and the addition is trivial.
- Do not add external analytics, queues, background jobs, storage buckets, or edge functions unless a documented requirement cannot work without one.
- Show useful loading, empty, and error states. Avoid silent failures.

Suggested routes

Route names may follow the generated project conventions, but the product needs these destinations:

- `/login`
- `/onboarding` for create/join group
- `/group/:groupId` for today's problem and daily progress
- `/group/:groupId/progress` for the seven-day heatmap
- `/group/:groupId/problem/:problemId` for discussion
- `/group/:groupId/admin/problem` for create/edit

Protected pages redirect unauthenticated users to sign in. Non-members must receive no group data.

Build order

- 1 Connect Supabase and authentication.

- 2 Create schema, constraints, indexes, and Row Level Security policies.
- 3 Create/join group and membership checks.
- 4 Admin creates today's problem.
- 5 Member marks/unmarks completion.
- 6 Render daily count and member statuses from database records.
- 7 Render the seven-day heatmap from problems and completions.
- 8 Add minimal comments.
- 9 Add loading, empty, validation, and error states.
- 10 Verify mobile layout, access isolation, refresh persistence, and Vercel production build.

UI behavior

- Default to the current local date for today's problem while storing dates as a PostgreSQL date.
- Difficulty values: Easy, Medium, or Hard.
- External problem links open safely in a new tab.
- The completion button must clearly distinguish incomplete, saving, completed, and error states.
- Disable duplicate form submission while a request is pending.
- Confirm before deleting a problem because its completions and comments will also be removed.
- Empty states should tell the user what to do next.

Environment and deployment

- Use the environment-variable names produced by the Lovable project for the Supabase project URL and public anonymous key.
- Configure the production Vercel URL as an allowed Supabase Auth redirect URL.
- Confirm client-side routing works on a direct page load in production.
- Do not expose private keys in the repository or Vercel client bundle.

Verification

- Run lint/type checks and the production build used by the generated stack.
- Test with at least two accounts: owner and member.
- Test a non-member attempting to access the group.
- Test at approximately 390 px mobile width and a desktop width.
- Complete every item in `ACCEPTANCE_CRITERIA.md` before sharing.

Definition of prototype complete

The app is complete when the required core flow works on the deployed Vercel URL, Supabase policies prevent cross-group access and unauthorized writes, and the acceptance criteria pass. Visual extras and future features do not count toward completion.